

使用具備 AI 功能的 BigQuery DataFrames 套件， 從結構化與非結構化資料取得洞察資訊

來源：<https://codelabs.developers.google.com/bigquery-dataframes-clustering-unstructured-data?hl=zh-tw>

步驟數：9

在本研究室中，您將使用 BigQuery Studio 中的 Python 筆記本，透過 BigQuery DataFrames 從非結構化資料中取得洞察資料。

目錄

1. [1. 總覽](#)
2. [2. 需求條件](#)
3. [3. 讀取公開資料集](#)
4. [4. 探索資料](#)
5. [5. 計算嵌入](#)
6. [6. 使用文字嵌入進行叢集](#)
7. [7. 歸納叢集重點](#)
8. [8. 清除所用資源](#)
9. [9. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽

在本實驗室中，您將使用 BigQuery Studio 的 Python 筆記本中的 BigQuery DataFrames，透過 Python 取得資料洞察。運用 Google 的生成式 AI 分析非結構化文字資料，並以視覺化方式呈現。

您將建立 Python 筆記本，對公開的顧客申訴資料庫進行分類和摘要。這項技術可調整，適用於任何非結構化文字資料。

目標

在本實驗室中，您將瞭解如何執行下列工作：

- 在 BigQuery Studio 啟用及使用 Python 筆記本
 - 使用 BigQuery DataFrames 套件連結至 BigQuery
 - 使用 BigQuery ML 並連結至 Vertex AI 中的文字嵌入端點，從非結構化文字資料建立嵌入
 - 使用 BigQuery ML 叢集嵌入
 - 透過 BigQuery ML 使用 LLM 匯總叢集
-

步驟 2 · 約 0 分鐘

2. 需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 已啟用計費功能的 Google Cloud 專案

事前準備

如要按照本程式碼研究室的說明操作，您需要啟用 BigQuery Studio 的 Google Cloud 專案，並連結帳單帳戶。

1. 在 [Google Cloud 控制台](#) 的專案選擇器頁面中，選取或建立 Google Cloud [專案](#)
2. 確保您的 Google Cloud 專案有啟用計費服務。瞭解如何[檢查專案是否已啟用計費功能](#)
3. 按照操作說明[啟用 BigQuery Studio 資產管理功能](#)。

準備 BigQuery Studio

建立空白筆記本，並連線至執行階段。

1. 前往 Google Cloud 控制台的「BigQuery Studio」。
2. 按一下「+」按鈕旁的 ▼。
3. 選取「Python 筆記本」。
4. 關閉範本選取器。
5. 選取「+ Code」建立新的程式碼儲存格。
6. 從程式碼儲存格安裝最新版 BigQuery DataFrames 套件。輸入下列指令：

□□□

```
%pip install --upgrade bigframes --quiet
```

點選「□」按鈕或按下 *Shift + Enter* 鍵，執行程式碼儲存格。

步驟 3 · 約 0 分鐘

3. 讀取公開資料集

在新程式碼儲存格中執行下列指令，初始化 BigQuery DataFrames 套件：

```
import bigframes.pandas as bpd

bpd.options.bigquery.ordering_mode = "partial"
```

注意：在本教學課程中，我們使用實驗性的「部分排序模式」，搭配類似 pandas 的篩選條件時，可提高查詢效率。需要嚴格排序或索引的某些 pandas 功能可能無法運作。

消費者申訴資料庫

[消費者申訴資料庫](#)是透過 [Google Cloud](#) 的公開資料集計畫，在 BigQuery 上提供。這是美國消費者金融保護局收集的消費金融產品和服務申訴資料。

在 BigQuery 中，查詢 [bigquery-public-data.cfbp_complaints.complaint_database](#) 資料表，分析消費者申訴資料庫。使用 `bigframes.pandas.read_gbq()` 方法，從查詢字串或資料表 ID 建立 DataFrame。

在新的程式碼儲存格中執行下列程式碼，建立名為「feedback」的 DataFrame：

```
feedback = bpd.read_gbq(
    "bigquery-public-data.cfbp_complaints.complaint_database"
)
```

探索 DataFrame 的基本資訊

使用 `DataFrame.peek()` 方法下載少量資料樣本。

執行這個儲存格：

```
feedback.peek()
```

預期的輸出內容：

	date_received	product ...	timely_response	consumer_disputed	complaint_id
0	2014-03-05	Bank account or service ...	True	False	743665
1	2014-01-21	Bank account or service ...	True	False	678608
2	2020-12-31	Debt collection ...	True	<NA>	4041190
3	2014-02-12	Debt collection ...	True	False	714350
4	2015-02-23	Debt collection ...	True	False	1251358

注意：`head()` 需要排序，且如果您想將資料樣本視覺化，`head()` 的效率通常不如 `peek()`。

與 pandas 相同，使用 `DataFrame.dtypes` 屬性即可查看所有可用資料欄及其對應的資料類型。這些資料會以與 pandas 相容的方式公開。

執行這個儲存格：

```
feedback.dtypes
```

預期的輸出內容：

```
date_received      date32[day][pyarrow]
product            string[pyarrow]
subproduct         string[pyarrow]
issue              string[pyarrow]
subissue           string[pyarrow]
consumer_complaint_narrative string[pyarrow]
company_public_response string[pyarrow]
company_name       string[pyarrow]
state              string[pyarrow]
zip_code           string[pyarrow]
tags               string[pyarrow]
consumer_consent_provided string[pyarrow]
submitted_via      string[pyarrow]
date_sent_to_company date32[day][pyarrow]
company_response_to_consumer string[pyarrow]
timely_response    boolean
consumer_disputed  boolean
complaint_id       string[pyarrow]
dtype: object
```

`DataFrame.describe()` 方法會查詢 `DataFrame` 中的一些基本統計資料。由於這個 `DataFrame` 不含任何數值資料欄，因此會顯示非空值計數和不重複值數量的摘要。

執行這個儲存格：

```
# Exclude some of the larger columns to make the query more efficient.
feedback.drop(columns=[
    "consumer_complaint_narrative",
    "company_public_response",
    "company_response_to_consumer",
]).describe()
```

預期的輸出內容：

	product	subproduct	issue	subissue	company_name	state	...	timely_response
	consumer_disputed	complaint_id						
count	3458906	3223615	3458906	2759004	3458906	3417792	...	3458906
	768399	3458906						
nunique	18	76	165	221	6694	63	...	2
	2	3458906						

步驟 4 · 約 0 分鐘

4. 探索資料

在深入瞭解實際申訴內容之前，請先使用 `DataFrame` 上的類似 `pandas` 的方法，將資料視覺化。

以視覺化方式呈現 `DataFrame`

有幾種內建的視覺化方法，例如 `DataFrame.plot.hist()`。由於這個 `DataFrame` 大多包含字串和布林值資料，我們可以先進行一些匯總，進一步瞭解各種資料欄。

計算各州收到的申訴數量。

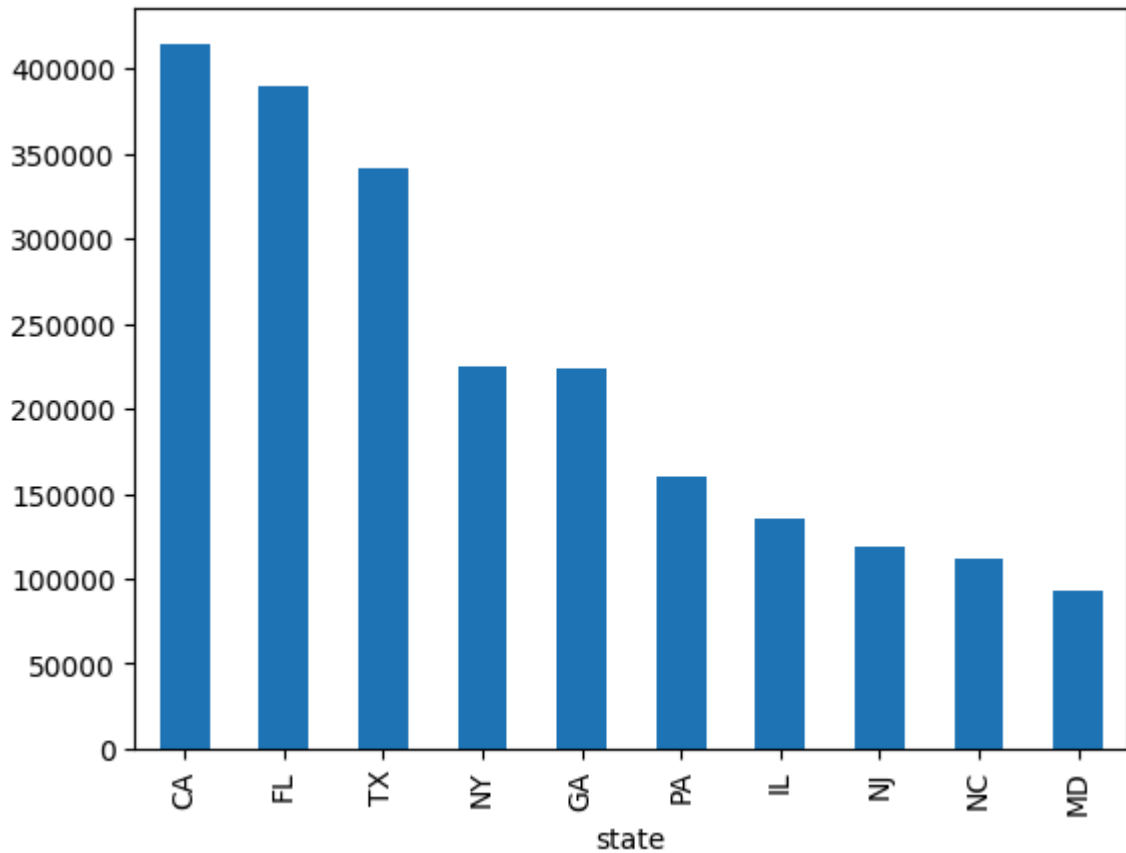
```
complaints_by_state = (  
    feedback.groupby(  
        "state", as_index=False,  
    ).size()  
    .rename(columns={"size": "total_complaints"})  
    .sort_values(by="total_complaints", ascending=False)  
)
```

使用 `DataFrame.to_pandas()` 方法將其轉換為 `pandas DataFrame`。

```
complaints_pd = complaints_by_state.head(10).to_pandas()
```

對這個下載的 `DataFrame` 使用 `pandas` 視覺化方法。

```
complaints_pd.plot.bar(x="state", y="total_complaints")
```

與其他資料集彙整

先前您查看的是各州的申訴案件數，但這樣會遺漏重要背景資訊。有些州的人口比其他州多。與人口資料集 (例如[美國人口調查局的美國社區問卷調查](#)和 [bigquery-public-data.geo_us_boundaries.states](#) 資料表) 聯結。

```
us_states = bpd.read_gbq("bigquery-public-data.geo_us_boundaries.states")
us_survey = bpd.read_gbq("bigquery-public-data.census_bureau_acs.state_2020_5yr")

# Ensure there are leading 0s on GEOIDs for consistency across tables.
us_states = us_states.assign(
    geo_id=us_states["geo_id"].str.pad(2, fillchar="0")
)

us_survey = us_survey.assign(
    geo_id=us_survey["geo_id"].str.pad(2, fillchar="0")
)
```

美國社區調查會以 GEOID 識別各州。使用州/省資料表加入，即可依兩個字母的州/省代碼取得人口資料。

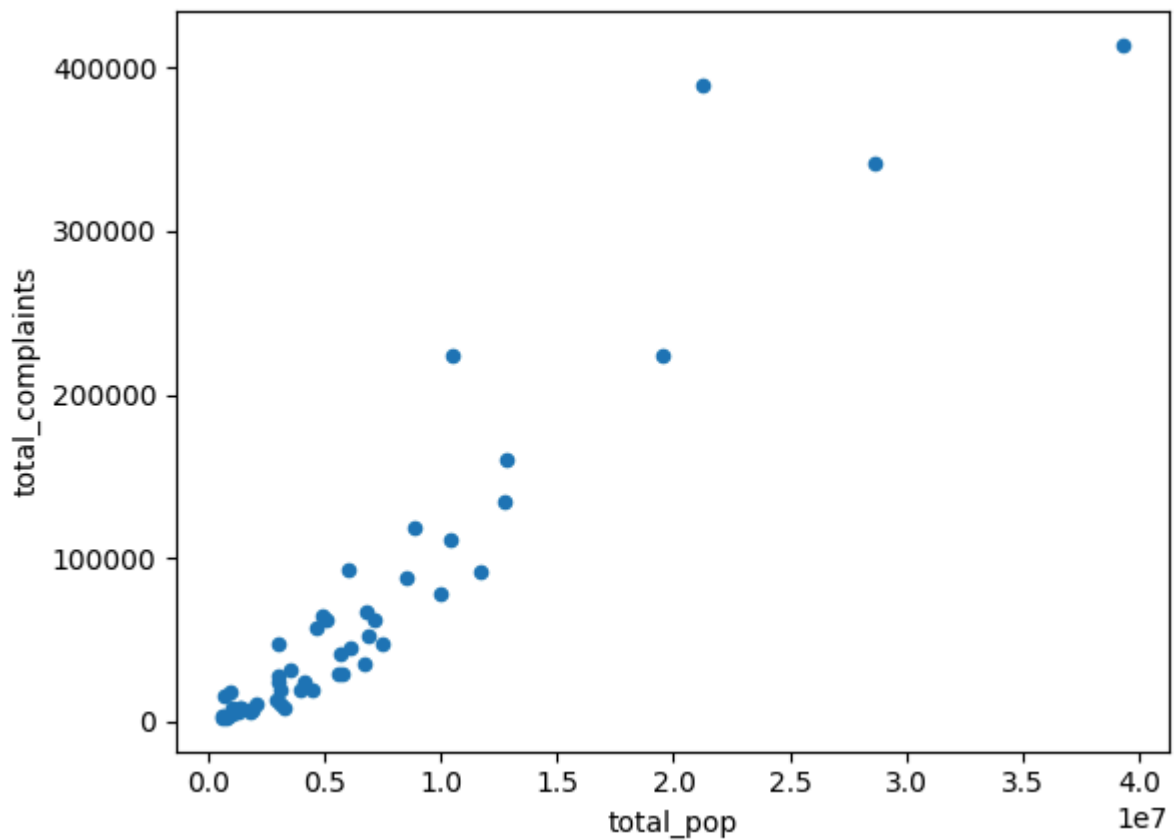
```
pops = us_states.set_index("geo_id")[["state"]].join(
    us_survey.set_index("geo_id")[["total_pop"]]
)
```

現在，請將這項資料與客訴資料庫合併，比較人口數與客訴數量。

```
complaints_and_pops = complaints_by_state.set_index("state").join(
    pops.set_index("state")
)
```

建立散佈圖，比較各州人口數與申訴件數。

```
(
    complaints_and_pops
    .to_pandas()
    .plot.scatter(x="total_pop", y="total_complaints")
)
```



比較人口數和申訴件數時，有幾個州似乎是離群值。請讀者自行練習繪製點標籤，以找出這些內容。同樣地，請針對可能的原因提出一些假設 (例如不同的人口統計資料、不同數量的金融服務公司等)，並進行測試。

步驟 5 · 約 0 分鐘

5. 計算嵌入

重要資訊通常隱藏在文字、音訊或圖片等非結構化資料中。在這個範例中，申訴資料庫中的許多實用資訊都包含在申訴的文字內容中。

AI 和傳統技術 (例如情緒分析、「詞袋」和 word2vec) 可從非結構化資料中擷取部分量化資訊。最近，與 LLM 密切相關的「向量嵌入」模型可建立一連串浮點數，代表文字的語意資訊。

選取資料庫子集

執行向量嵌入模型比其他作業更耗用資源。為減少費用和配額問題，請選取資料子集，以完成本教學課程的其餘部分。

```
import bigframes.pandas as bpd

bpd.options.bigquery.ordering_mode = "partial"

feedback = bpd.read_gbq(
    "bigquery-public-data.cfpb_complaints.complaint_database"
)

# Note: if not using ordering_mode = "partial", you must specify these in read_gbq
# for these to affect query efficiency.
# feedback = bpd.read_gbq(
#     "bigquery-public-data.cfpb_complaints.complaint_database",
#     columns=["consumer_complaint_narrative"],
#     filters= [
#         ("consumer_complaint_narrative", "!=", ""),
#         ("date_received", "==", "2022-12-01")]
# )

feedback.shape
```

2022 年 12 月 1 日提交的申訴案件約有 1,000 件，相較於資料庫總共近 350 萬列資料 (請與 `feedback.shape` 確認)，只選取 2022 年 12 月 1 日的資料，且只選取 `consumer_complaint_narrative` 欄。

```
import datetime

feedback = feedback[
    # Filter rows by passing in a boolean Series.
    (feedback["date_received"] == datetime.date(2022, 12, 1))
    & ~(feedback["date_received"].isnull())
    & ~(feedback["consumer_complaint_narrative"].isnull())
    & (feedback["consumer_complaint_narrative"] != "")
    & (feedback["state"] == "CA")

    # Uncomment the following if using free credits for a workshop.
    # Billing accounts with free credits have limited Vertex AI quota.
    # & (feedback["product"] == "Mortgage")
][
    # Filter columns by passing in a list of strings.
    ["consumer_complaint_narrative"]
]

feedback.shape
```

pandas 的 `drop_duplicates` 方法需要資料列的總排序，因為該方法會嘗試選取第一個或最後一個相符的資料列，並保留與其相關聯的索引。

請改用 `groupby` 方法呼叫來彙整資料，以移除重複的資料列。

```
feedback = (
    feedback.groupby("consumer_complaint_narrative", as_index=False)
    .size()
)[["consumer_complaint_narrative"]]

feedback.shape
```

生成嵌入項目

BigQuery DataFrames 會透過 [TextEmbeddingGenerator](#) 類別產生嵌入向量。這是以 [ML.GENERATE_EMBEDDING](#) 方法為基礎，在 BigQuery ML 中呼叫 Vertex AI 提供的[文字嵌入模型](#)。

```
from bigframes.ml.llm import TextEmbeddingGenerator

embedding_model = TextEmbeddingGenerator(
    model_name="text-embedding-004"
)
feedback_embeddings = embedding_model.predict(feedback)
```

請參閱嵌入項目的外觀。這些向量代表文字嵌入模型所理解的文字語意。

```
feedback_embeddings.peek()
```

預期的輸出內容：

```
ml_generate_embedding_result \
0 [ 7.36380890e-02  2.11779331e-03  2.54309829e-...
1 [-1.10935252e-02 -5.53950183e-02  2.01338865e-...
2 [-7.85628427e-03 -5.39347418e-02  4.51385677e-...
3 [ 0.02013054 -0.0224789 -0.00164843  0.011354...
4 [-1.51684484e-03 -5.02693094e-03  1.72322839e-...
```

這些向量具有許多維度。請看單一嵌入向量：

```
feedback_embeddings["ml_generate_embedding_result"].peek().iloc[0]
```

嵌入內容生成作業適用「部分成功」合約。這表示部分資料列可能發生錯誤，因此無法產生嵌入內容。'ml_generate_embedding_status' 欄會顯示錯誤訊息。空白表示沒有錯誤。

篩選嵌入內容，只納入未發生錯誤的資料列。

```
mask = feedback_embeddings["ml_generate_embedding_status"] == ""
valid_embeddings = feedback_embeddings[mask]
valid_embeddings.shape
```

6. 使用文字嵌入進行叢集

現在，請使用 k-means 將嵌入分組。在本示範中，請使用任意數量的群組 (又稱形心)。如要提供正式版品質的解決方案，應使用「[輪廓法](#)」等技術調整質心數量。

```
from bigframes.ml.cluster import KMeans

num_clusters = 5
cluster_model = KMeans(n_clusters=num_clusters)
cluster_model.fit(valid_embeddings["ml_generate_embedding_result"])
```

```
clusters = cluster_model.predict(valid_embeddings)
clusters.peek()
```

移除所有嵌入失敗的項目。

```
mask = clusters["ml_generate_embedding_status"] == ""
clusters = clusters[mask]
```

查看每個質心的留言分布情形。

```
clusters.groupby("CENTROID_ID").size()
```

7. 歸納叢集重點

提供與每個質心相關的留言，並要求 Gemini 歸納出抱怨內容。提示工程是新興領域，但網路上有許多實用範例，例如 <https://www.promptingguide.ai/>。

```
from bigframes.ml.llm import GeminiTextGenerator

preamble = "What is the main concern in this list of user complaints:"
suffix = "Write the main issue using a formal tone."

# Now let's sample the raw comments and get the LLM to summarize them.
prompts = []
for centroid_id in range(1, num_clusters + 1):
    cluster = clusters[clusters["CENTROID_ID"] == centroid_id]
    comments = "\n".join(["- {}".format(x) for x in cluster.content.peek(40)])
    prompts.append("{}:\n{}\n{}".format(preamble, comments, suffix))

prompt_df = bpd.DataFrame(prompts)
gemini = GeminiTextGenerator(model_name="gemini-1.5-flash-001")
issues = gemini.predict(X=prompt_df, temperature=0.0)
issues.peak()
```

使用 Gemini 根據摘要撰寫報告。

```
from IPython.display import display, Markdown

prompt = "Turn this list of issues into a short, concise report:"
for value in issues["ml_generate_text_llm_result"]:
    prompt += "- {}".format(value)
prompt += "Using a formal tone, write a markdown text format report."

summary_df = bpd.DataFrame([prompt])
summary = gemini.predict(X=summary_df, temperature=0.0)

report = (summary["ml_generate_text_llm_result"].values[0])
display(Markdown(report))
```

步驟 8 · 約 0 分鐘

8. 清除所用資源

如果您是為了這個教學課程建立新的 Google Cloud 專案，可以[刪除專案](#)，以免系統向您收取建立資料表或其他資源的額外費用。

步驟 9 · 約 0 分鐘

9. 恭喜！

您已使用 BigQuery DataFrames 分析結構化和非結構化資料。您已探索 Google Cloud 的公開資料集、BigQuery Studio 中的 Python 筆記本、BigQuery ML、Vertex AI，以及 BigQuery Studio 的自然語言轉 Python 功能。太棒了！

後續步驟

- 請嘗試在筆記本中生成 [Python 程式碼](#)。BigQuery Studio 中的 Python 筆記本採用 Colab Enterprise 技術。提示：要求生成測試資料相當實用。
 - 在 GitHub 上探索 [BigQuery DataFrames 的範例筆記本](#)。
 - 排定在 BigQuery Studio 中執行筆記本的時間。
 - 部署 [搭配 BigQuery DataFrames 的遠端函式](#)，將第三方 Python 套件與 BigQuery 整合。
-